

Examen-1.pdf



qilnx



Estructuras de Datos Avanzadas



3º Doble Grado en Ingeniería Informática e Ingeniería del Software



**Escuela Técnica Superior de Ingeniería Informática.
Campus de Móstoles
Universidad Rey Juan Carlos**



[Accede al documento original](#)



**TECNOLOGÍA
a PRECIO de
ESTUDIANTE**

VERANO como ESTRELLA
DEL ROCK



webuy.com
CLIC AQUÍ

OJALÁ
ESTE TEMA
TE ENTRASE
COMO UNA
BUENA PIZZA
UN VIERNES
NOCHE.



MEDIANAS 2 INGREDIENTES

2X **5'50** €
c/u
RECOGER
CÓD: WLAH5

3X **7** €
c/u
DOMICILIO
CÓD: WLAH7



PIDE
AQUÍ

Domino's®

Consulta condiciones,
suplementos y tiendas
adheridas en la web. Se
podrá solicitar el carnet
de estudiante para apli-
car las promociones.
Exclusivo para menores
de 25 años. Válida hasta
el 31/12/2026.

Estructuras de Datos Avanzadas Examen Ordinario 20 de enero de 2022

- La duración del examen es de 2 horas.
- Desde el campus virtual hay que descargar un archivo ZIP que contiene el esqueleto de los ejercicios del examen así como los test unitarios correspondientes. Se recomienda crear un proyecto de Netbeans vacío; copiar dentro de la carpeta del proyecto la carpeta src y la carpeta test proporcionadas; añadir las librerías JUnit y Hamcrest; y finalmente cerrar y volver a abrir Netbeans.
- El alumno debe generar un fichero ZIP con el directorio en el que esté el código del alumno (examenEnero) y subirá el fichero ZIP a la actividad examen práctico del aula virtual asegurándose de que entrega únicamente el código desarrollado durante la prueba.

Ejercicio 1 [4 puntos]

La clase `MinimumSuccessorTree` implementa en esencia un árbol de búsqueda binario en el que el usuario tiene acceso a las operaciones que devuelven el mínimo y el sucesor de un nodo dado. Esta clase está incompleta y se desea completar implementando los métodos `minimum()`, `minimum(Position<E> pos)` e `iterator()`.

En este ejercicio solo se deben modificar las clases `MinimumSuccessorTree`, `MaximumPredecesorTree` e `InorderMinimumSuccessorTreeIterator` respetando siempre las cabeceras definidas.

- [0.5 puntos]** En la clase `MinimumSuccessorTree` implementar el método `minimum(Position <E> pos)` que devuelve el `position` con el valor mínimo del subárbol que tiene raíz en "pos".
- [0.5 puntos]** En la clase `MinimumSuccessorTree` implementar el método `minimum()` que devuelve el `position` con el valor mínimo del árbol.
- [0.5 puntos]** En la clase `MinimumSuccessorTree` implementar el método `iterator()`, que proporciona un `iterator` `InorderMinimumSuccessorTreeIterator` (leer apartado siguiente antes de resolver este).
- [1.25 puntos]** Implementa la clase `InorderMinimumSuccessorTreeIterator`. Es posible implementar un recorrido inorden sobre un árbol binario de búsqueda de forma alternativa a la vista en clase utilizando únicamente los métodos `minimum` y `successor`. Se pide implementar esta versión alternativa de recorrido inorden. No se permite modificar las cabeceras de las funciones o de la clase y se deberán utilizar los métodos proporcionados en la clase `MinimumSuccessorTree`.

WUOLAH

- e) **[1.25 punto]** Por último, se desea convertir el `MinimumSuccessorTree` en un `MaximumPredecessorTree`. Este árbol binario de búsqueda es similar al `MinimumSuccessorTree` pero con la diferencia de que ordena los valores de mayor a menor. Es decir, el hijo izquierdo de un nodo almacena un valor mayor que su padre y este, a su vez, almacena un valor mayor que su hijo derecho. Si se realiza un recorrido inorden sobre este árbol con la clase `InorderBinaryTreeIterator` se obtiene una secuencia de nodos con valores ordenados de forma descendente. Por este motivo, las operaciones `minimum` y `successor` pasan a ser `maximum` y `predecessor`. Se pide completar la clase `MaximumPredecessorTree` para que se comporte como se ha descrito, así como los métodos `maximum()` y `máximo(Position<E> pos)`. No se debe modificar la clase padre `MaximumPredecessorAbstractTree` (Nótese que en este apartado no hay que implementar un iterador ya que se usa el `InorderBinaryTreeIterator` para los test).

Ejercicio 2 [6 puntos]

El estudio de desarrollo EDA_URJC quiere desarrollar una aplicación para gestionar una liga de baloncesto. Dicha aplicación debe permitir al usuario almacenar, modificar y consultar los equipos que compiten en la liga, así como los partidos que se disputan. Se pide implementar las clases `BasketballLeague` y `Team` que se encuentran en la carpeta con el material del examen. Las clases `Game` y `Player` ya se proporcionan implementadas, aunque se permite añadir funcionalidad adicional a estas si fuese necesario. Se valorará la elección de las estructuras de datos más adecuadas para almacenar los distintos tipos de datos. A continuación, se detallan las funcionalidades requeridas. Se recomienda leer detenidamente todos los subapartados antes de comenzar con su implementación.

- a. **[0.75 puntos]** Implementa la clase `Team`. Cada equipo debe tener un nombre asociado y estará compuesto por varios jugadores de la clase `Player`. Un jugador no puede pertenecer a dos equipos al mismo tiempo. Por último, una vez implementada esta clase se debe descomentar el método `toString()` de la clase `Game` para poder ejecutar los test.
- b. Implementa la clase `BasketballLeague` que se encarga de gestionar los equipos que componen la liga y los partidos.
 - I. **[0.25 puntos]** Se debe acompañar la clase `BasketballLeague` de un comentario con una descripción de las estructuras de datos elegidas y una justificación. No hacer este apartado invalidará el apartado (b) completo.
 - II. **[0.25 puntos]** Implementa el método `insertTeam()` que inserta un nuevo equipo en la liga.
 - III. **[0.5 puntos]** Implementa el método `insertGame(Team homeTeam, Team awayTeam, Date time, String result)` que inserta un nuevo partido en la liga. Este método recibe dos equipos, uno como local y otro como visitante (es importante determinar correctamente el rol de cada equipo), una fecha y un resultado.
 - IV. **[0.5 puntos]** Implementa los métodos `listTeams()` y `listGames()` que devuelven una lista con todos los equipos y todos los partidos respectivamente.
 - V. **[0.75 puntos]** Implementa el método `getPlayer(Team t, Integer number)` que devuelve un jugador dado su equipo y su dorsal en tiempo $O(1)$.

- VI. **[0.75 puntos]** Implementa el método `listGamesBetween(Team t1, Team t2)` que devuelve una lista con todos los partidos en los que juega el equipo t1 contra el t2 en la liga.
- VII. **[0.75 puntos]** Implementa el método `listHomeGames(Team t)` que devuelve una lista con todos los partidos que tiene el equipo t como equipo local en la liga.
- VIII. **[1.5 punto]** Implementa el método `transferPlayer(Team t1, Player p1, Team t2, Player p2)` que recibe dos jugadores (p1 y p2) con sus respectivos equipos (t1 y t2) y los traspasa. Es decir, como resultado el jugador p1 pasará a ser miembro del equipo t2 y el jugador p2 pasará al equipo t1. Ten en cuenta que en un mismo equipo no puede haber dos jugadores con el mismo número.

No se permite modificar las cabeceras de las clases ni de los métodos previamente definidos. Sí se permite implementar métodos y clases adicionales.

Se recuerda que en una competición tipo liga todos los equipos se enfrentan a todos los equipos dos veces, una como locales y otra como visitantes.